

Spectral Processing in Pd

Anxe Faraldo
anxefaraldo@gmail.com

Fourier Theorem

Fourier theorem

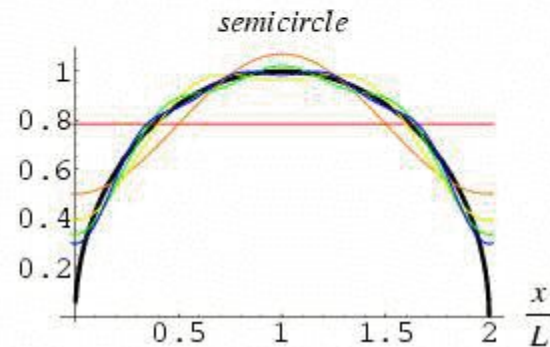
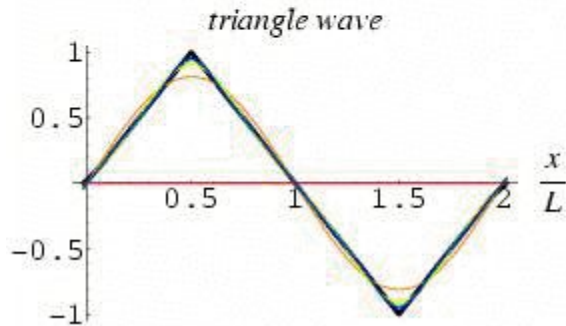
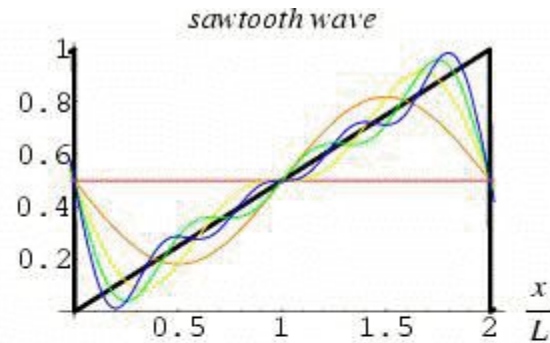
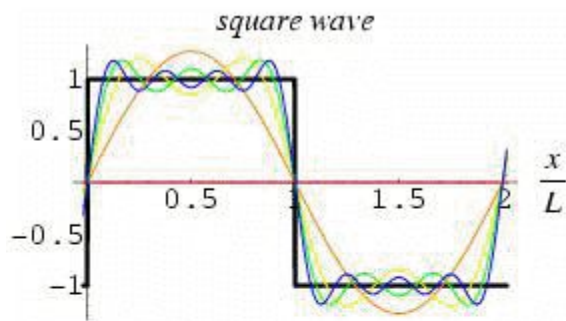
“Any periodic function $f(x)$ which is reasonably continuous may be expressed as the sum of a series of **sine** or **cosine** components”



Any periodic signal can be constructed with sinusoids!!!

Waveforms and Sinusoids

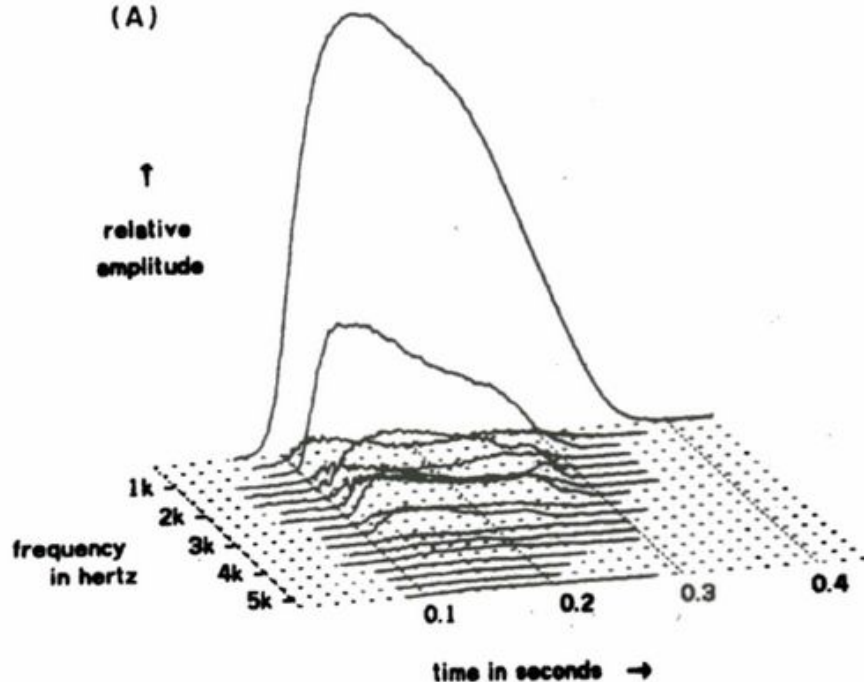
Any periodic signal can be constructed with sinusoids



Sinusoidal Component Analysis

An oboe

(A)



A trumpet

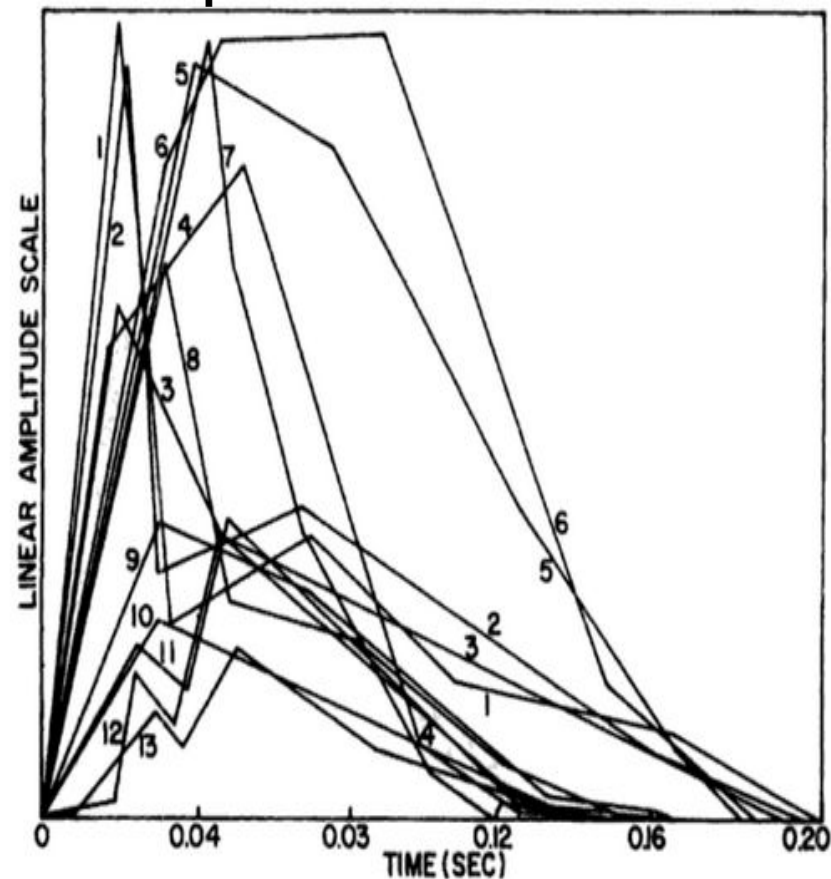
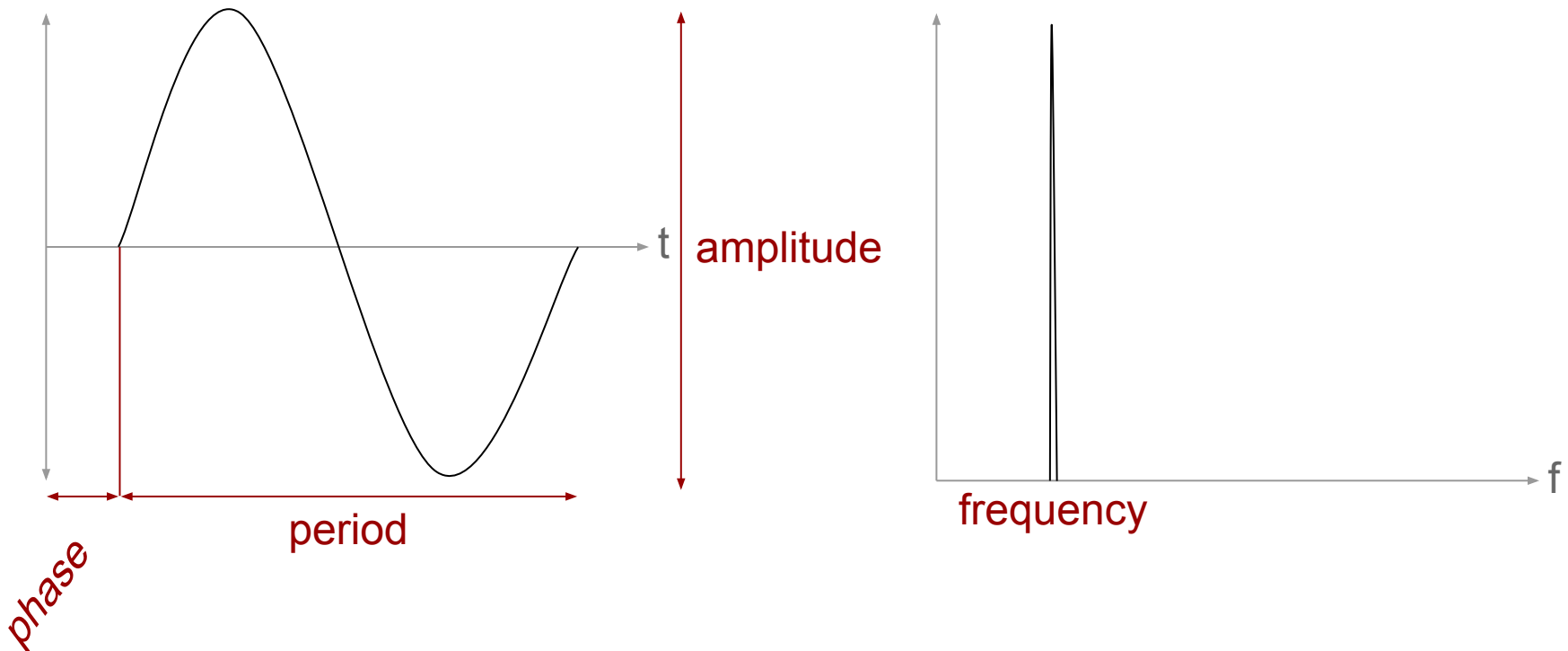


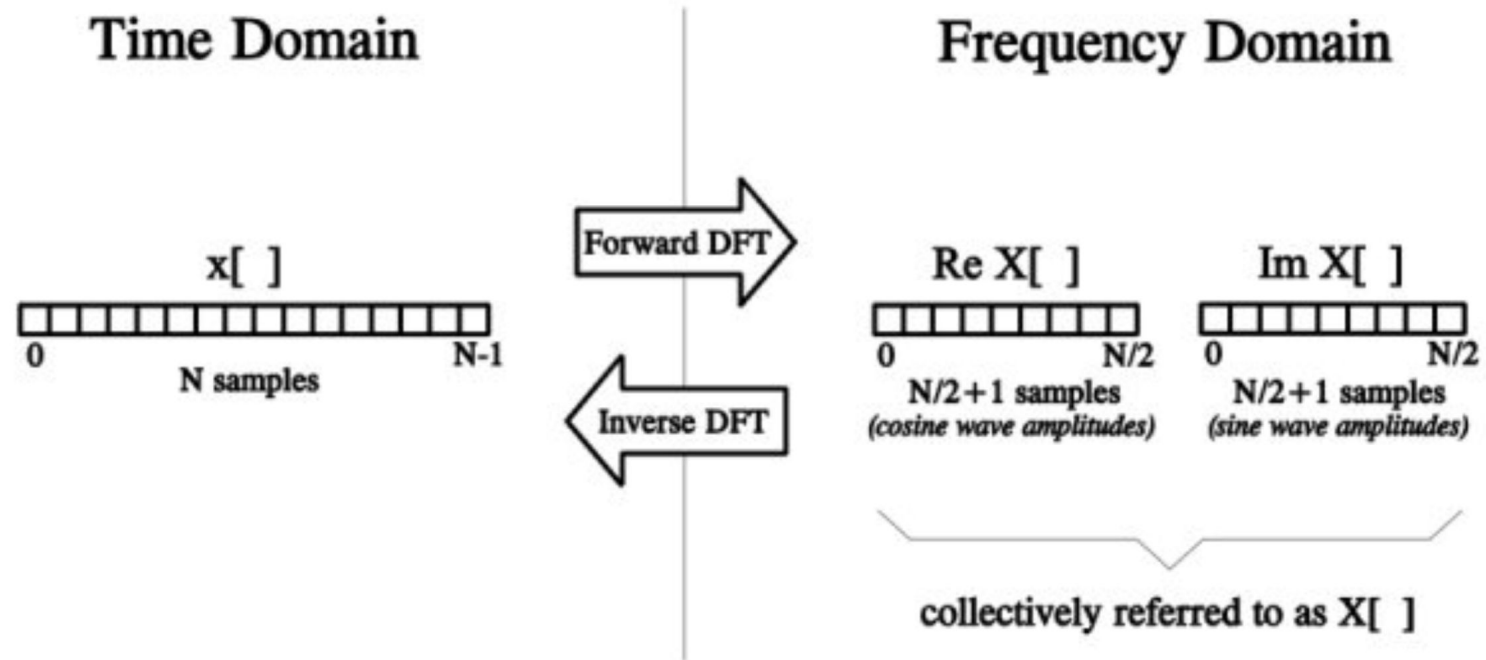
FIGURE 3 Line-segment functions that approximate the evolution in time of 13 harmonics of a D₄ trumpet tone lasting 0.2 sec. Functions like these, obtained by analysis of real tones, have been used to control the harmonic amplitudes of synthetic tones (Risset & Mathews, 1969).

Time vs. Frequency Domain

- Signals can be represented at time or frequency domains
 - Time ~ Waveform
 - Frequency ~ Spectrum
- Both domains are completely reversible



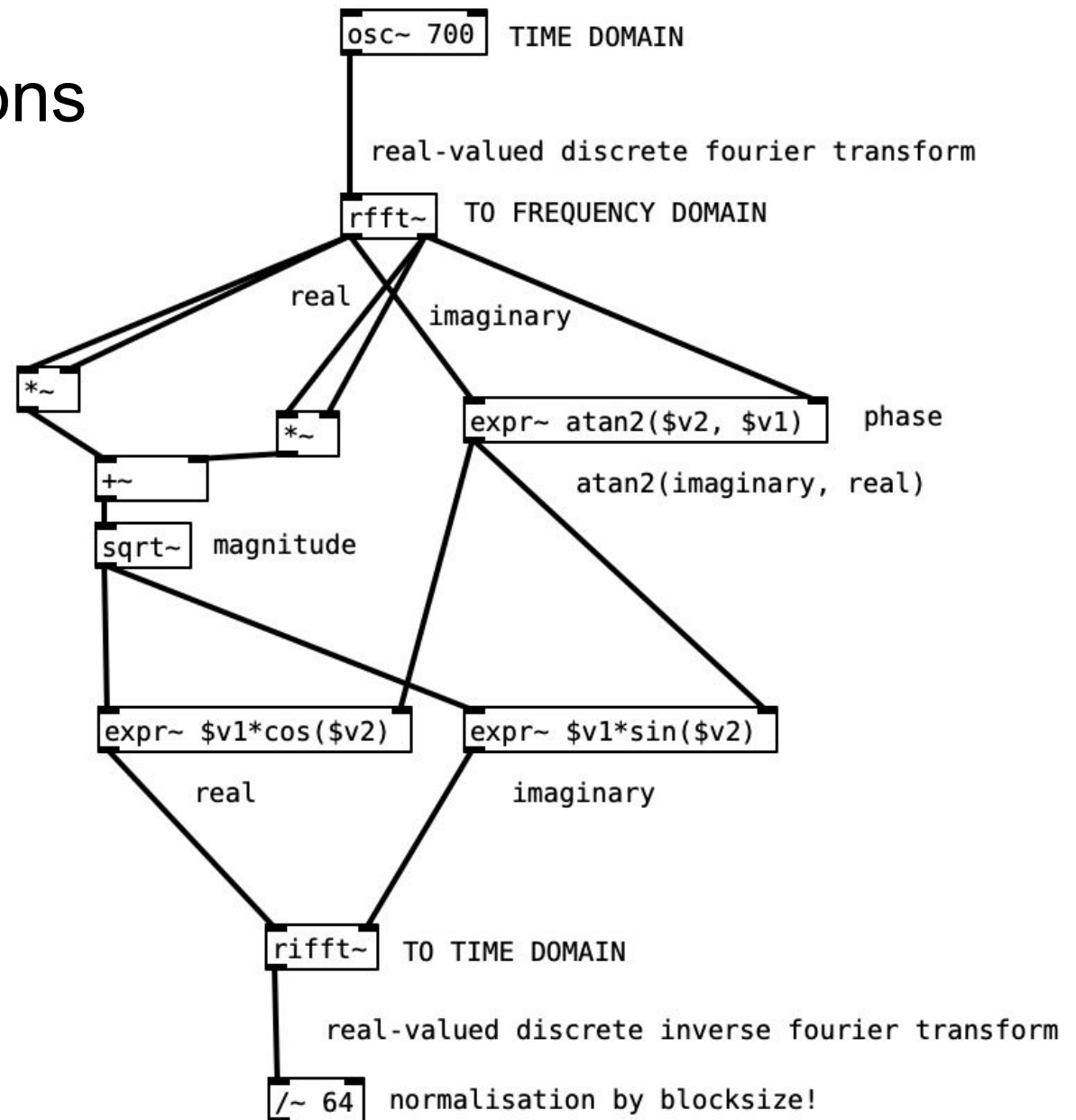
The Fourier Transform



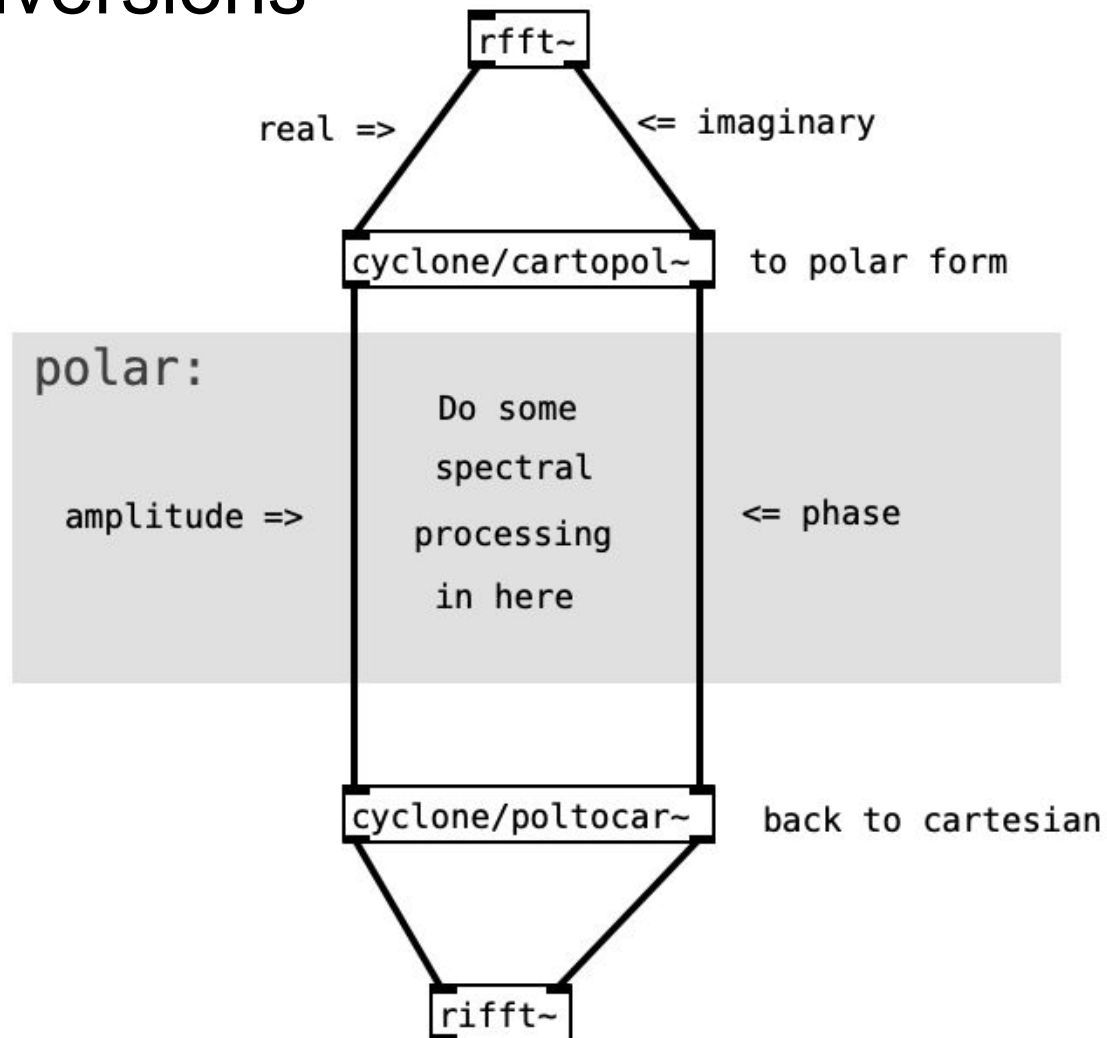
FFT in Pd

- [fft~] [ifft] forward and inverse FFT of complex signals
- FFT objects provide no normalisation (gain = N size)
- One transform per audio block!
- Blocksize must be a power of 2!
- We typically deal with **real-valued signals**
 - [rfft~] / [rifft~]

Unit Conversions



Unit Conversions



FFT Monitoring

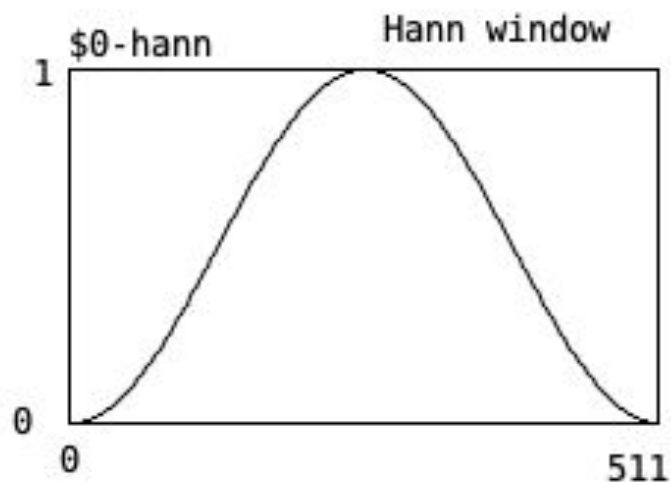
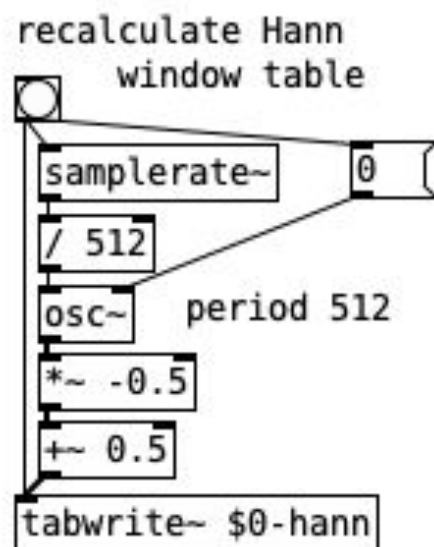
- [print~] object sends full audio blocks into the console
- [tabsend~] and [tabreceive~] manage audio blocks to/from tables on a block-by-block basis

FFT analysis size

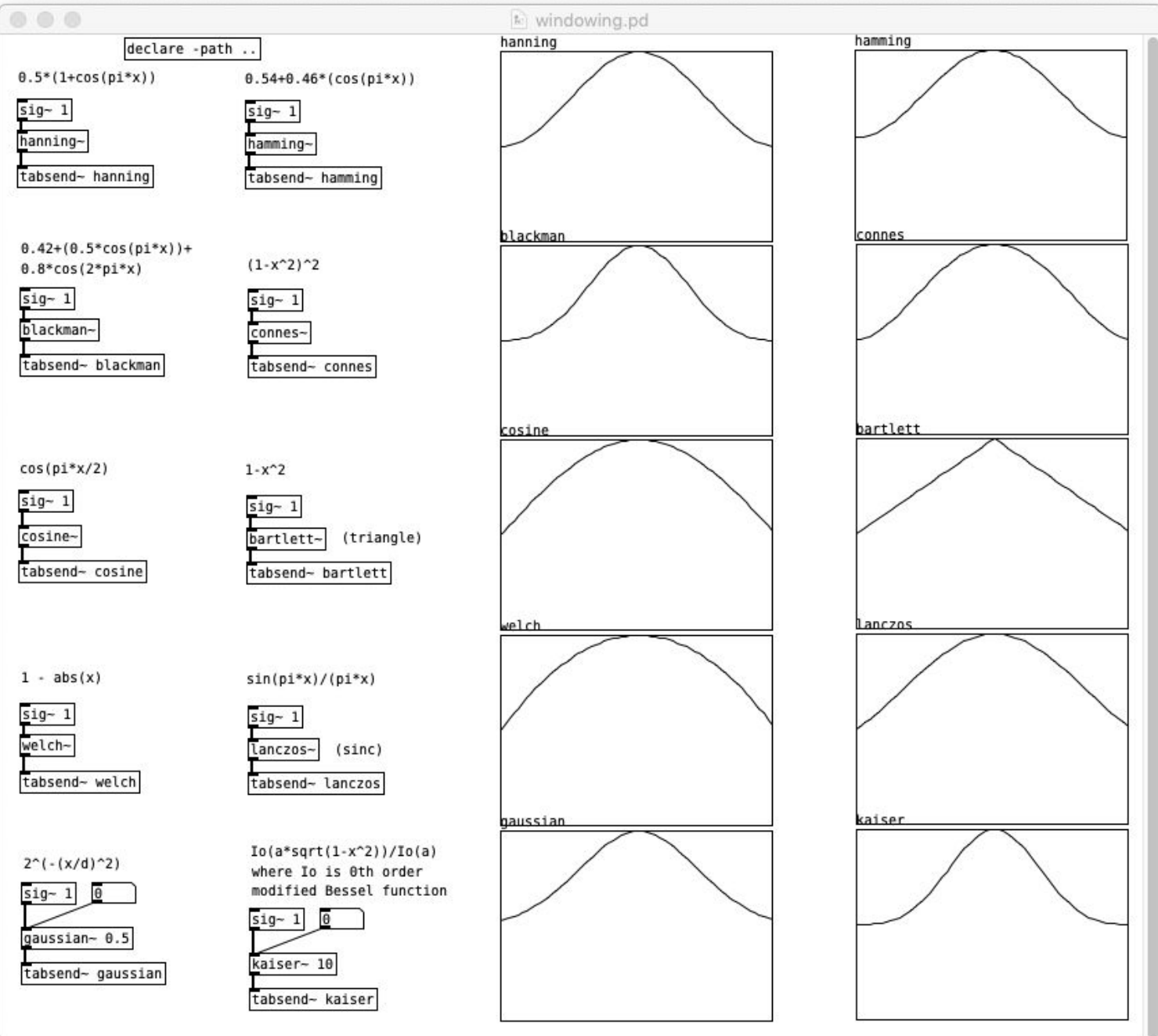
- 64 samples is a very small window for good spectral analysis. The frequency resolution will be very bad ($44.100 / 64 \sim 689.06\text{Hz}$).
- We will usually make a bigger FFT window (1024, 2048...) by changing the size of the Pd audio block.
- With a bigger FFT window, we will also need to do overlapping (for increasing time resolution), and thus use a non-rectangular window.
 - Otherwise it might result in unwanted spectral components

FFT windowing

- We typically calculate and use a Hann window
- The “Windowing” library provides other options



updated for Pd version 0.39



FFT windowing

- We manage the overlap factor in the [block~] object
- [inlet~] and [outlet~] manage reblocking and overlapping
- Window on a block basis with [tabreceive~]

`block~ 512 4`

block~ object specifies vector size of 512 and overlap four. This window now computes blocks of 512 samples at intervals of 128 samples computed on the parent patch.

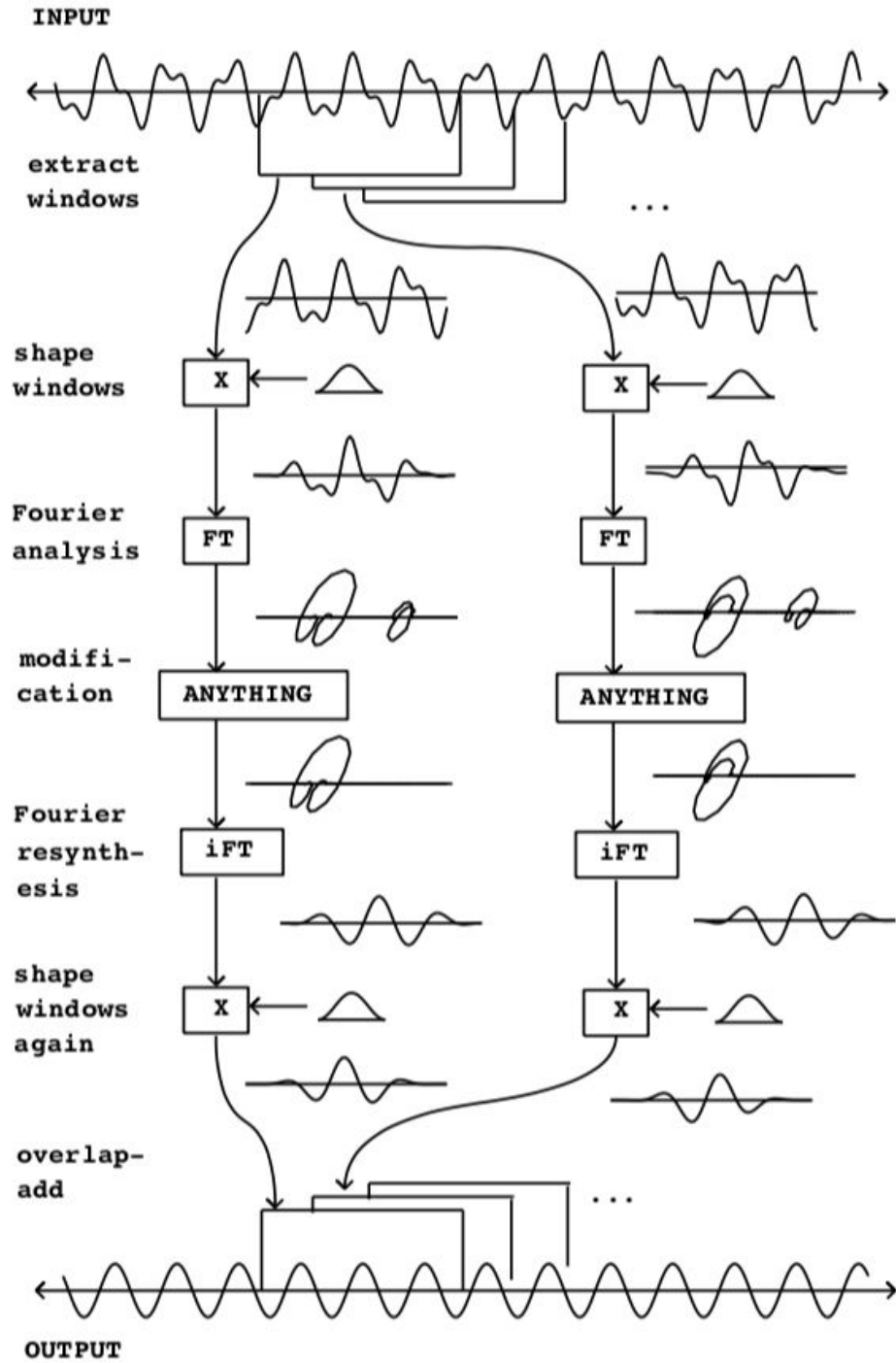
`inlet~`

The inlet~ now re-uses 3/4 of the previous block, along with the 128 new samples.

`tabreceive~ $0-hann`

window function as before.

`*~`



Common Spectral Techniques (1)

- Equalisation
- Denoising
 - By subtracting a mask of spectral background noise to the signal
- Dynamic Compression / Expansion
- Pitch / Noise Separation

Common Spectral Techniques (2)

- Timbre Morphing / Convolution:
 - Convolution in time domain equals pointwise multiplication in the frequency domain.
 - Typical uses include binaural panning, IR reverb & filter design, timbre stamping
 -
- Phase Vocoder
 - Frequency domain analysis/resynthesis algorithm which can operate temporal and pitch modifications on a signal. It originates in speech synthesis.
 - Typical uses include time stretching and pitch shifting

Expanding Spectral Processing in Pd

- [FFTease](#)

collection of Max/MSP and Pd objects implementing various forms of spectral sound processing. These include an additive-synthesis phase vocoder, noise reduction, cross synthesis, and more unusual forms of spectral processing.

- [TimbreID](#) (see next slides)

External library which computes many spectral descriptors parameters in a very convenient way. Besides analysis, it also offers methods for observing the evolution of a signal, describe signals generally, and classify sounds based on timbral characteristics.

- [\[convolve~\]](#) and other objects by Brent

Expanding Spectral Processing in Pd

- [\[earplug~\]](#), [\[cw_binaural~\]](#)
 - Convolution binaural filters using HRTF measurements.
- [\[EssentiaRT~\]](#)
 - Max and Pd
 - Exposes a lot of Essentia (MFCC, onsets, centroid etc.)
 - Can be a bit hard to setup...

Loading External Libraries in Pd...

- Single executable per object:
 - Copy the external library into your externals folder
 - Do not need be declared (e.g. [zexy/z~], [convolve~])
- Bundled libraries
 - Copy the external library into your externals folder:
 - Windows: *C:\Program Files\Common Files\Pd*
 - Linux: */usr/local/lib/pd-externals*
 - OSX: */Library/Pd*
 - Declare the library in startup options or with [declare]